

Fiche procédure php et php POO

Variable	Signification
<code>\$_FILES['screen-shot']['name']</code>	Contient le nom du fichier envoyé par le visiteur.
<code>\$_FILES['screen-shot']['type']</code>	Indique le type du fichier envoyé. Si c'est une image gif par exemple, le type sera <code>image/gif</code>
<code>\$_FILES['screen-shot']['size']</code>	Indique la taille du fichier envoyé.  Attention : cette taille est en octets. Il faut environ 1 000 octets pour faire 1 Ko, et 1 000 000 d'octets pour faire 1 Mo. La taille de l'envoi est limitée par PHP. Par défaut, impossible d'uploader des fichiers de plus de 8 Mo.
<code>\$_FILES['screen-shot']['tmp_name']</code>	Juste après l'envoi, le fichier est placé dans un répertoire temporaire sur le serveur en attendant que votre script PHP décide si oui ou non il accepte de le stocker pour de bon. Cette variable contient l'emplacement temporaire du fichier (c'est PHP qui gère ça).
<code>\$_FILES['screen-shot']['error']</code>	Contient un code d'erreur permettant de savoir si l'envoi s'est bien effectué ou s'il y a eu un problème et si oui, lequel. La variable vaut 0 s'il n'y a pas eu d'erreur.

Sécurisez un cookie avec les propriétés `httpOnly` et `secure`

Configurons les options `httpOnly` et `secure` sur le cookie.



Sans rentrer dans les détails, cela rendra votre cookie inaccessible en JavaScript sur tous les navigateurs qui supportent cette option (c'est le cas de tous les navigateurs récents). Cette option permet de réduire drastiquement les risques de faille XSS sur votre site, au cas où vous auriez oublié d'utiliser `htmlspecialchars` à un moment.

- Les variables superglobales sont des variables automatiquement créées par PHP. Elles se présentent sous la forme de tableaux (arrays) contenant différents types d'informations.
- Dans les chapitres précédents, nous avons découvert deux superglobales essentielles : `$_GET` (qui contient les données issues de l'URL) et `$_POST` (qui contient les données issues d'un formulaire).
- La superglobale `$_SESSION` permet de stocker des informations qui seront automatiquement transmises de page en page pendant toute la durée de visite d'un internaute sur votre site. Il faut au préalable activer les sessions en appelant la fonction `session_start()`.
- La superglobale `$_COOKIE` représente le contenu de tous les cookies stockés par votre site sur l'ordinateur du visiteur. Les cookies sont de petits fichiers que l'on peut écrire sur la machine du visiteur pour retenir par exemple son nom. On crée un cookie avec la fonction `setcookie()`.

Dans "programmation orientée objet", il y a le mot *objet* (appelez-moi Sherlock Holmes). En programmation, **objet** est le nom que l'on donne à l'élément :

- qui se trouve **dans une variable** ;
- dans lequel on peut **stocker des valeurs** ;
- et auquel on peut **demandeur de faire des actions**.

Pour faire simple, c'est un regroupement de valeurs et de fonctions.

Pour obtenir un objet, il faut demander au langage de le créer et de nous le donner pour qu'on puisse le manipuler. Pour faire ça, on précise au langage le nom de l'élément que l'on veut manipuler, c'est-à-dire **la classe**.



Qu'est-ce qu'une classe ?

Une **classe** est une définition qui contient le nom des propriétés qu'on pourra manipuler ainsi que des méthodes. Les **propriétés**, ce sont des variables internes à cette définition dans lesquelles on stocke des valeurs. Une classe a aussi des **méthodes** : il s'agit de fonctions internes à la classe. La classe détermine ce qu'il sera possible de faire avec l'objet.

Le fait de créer un objet à partir d'une classe s'appelle **instancier une classe**.

- En programmation orienté objet, une **classe** est un modèle ou un mode d'emploi tandis que les **objets** sont les objets créés à partir de ce modèle
 - Une classe possède un ensemble de variables, appelées **propriétés** ainsi qu'un ensemble de fonctions appelées **méthodes**.
 - Il est possible de créer un objet à partir d'une classe grâce au mot-clé `new` . On appelle cela une **instance**.
 - Pour accéder aux propriétés ou aux méthodes d'une classe, on utilise le symbole `->` .
-
- Les objets possèdent des **propriétés** et des **méthodes**
 - Pour désigner une instance, nous utilisons le mot clé `$this` , permettant de faire appel aux propriétés ou méthodes de notre objet au sein d'une autre méthode.
 - Vous réduisez la complexité d'utilisation des classes en privatisant les méthodes et les propriétés avec `private` .
 - Nous pouvons appliquer **le principe d'encapsulation**, en utilisant des getters et des setters, permettant d'accéder aux propriétés et de les modifier en s'assurant de garder des valeurs cohérentes.
 - En réduisant la complexité, vous réduisez le risque d'erreur.
 - En utilisant la staticité vous pouvez partager des valeurs, ou appeler des méthodes "utilitaires", n'ayant pas d'impact sur l'état de l'objet.
 - Il est possible et recommandé de créer des constantes pour les informations qui ne changent pas, grâce au mot-clé `const` .
 - Pour appeler une propriété ou méthode statique au sein d'une classe, il faut utiliser le mot clé `self` .
 - Il existe des méthodes dites magiques, qui sont appelées automatiquement par PHP. Elles commencent par `__` .
 - `__construct` est la méthode appelée lors de la création d'un objet ; `__destruct` lors de sa suppression de la mémoire.

→ : Lorsque vous utilisez l'héritage, depuis l'objet comme depuis la classe, vous pouvez accéder aux propriétés de la classe parente de la même manière qu'avant, avec la flèche

- En utilisant le mot clé **private** , vous empêchez quiconque autre que la classe courante de manipuler les propriétés et méthodes.
- En utilisant le mot clé **protected** , vous donnez l'autorisation à la classe courante ainsi qu'à ses enfants de manipuler les propriétés et méthodes 😊 ; mais pas aux éléments extérieurs à la classe, ou extérieurs aux objets de cette classe.
- Et comme nous l'avons vu précédemment, avec le mot-clé **public** , il est possible de manipuler la méthode ou la propriété depuis la classe, ses classes enfants et depuis l'extérieur.
- Autrement dit, pour modifier une propriété depuis un enfant, il faut qu'elle possède la visibilité **public** ou **protected** , ou qu'elle possède un mutateur.
- En utilisant le mot clé **abstract** , vous pouvez imposer à une classe d'être héritée.
- Une classe abstraite ne peut plus être instanciée seule, et peut contenir des méthodes abstraites.
- Une méthode abstraite doit être implémentée dans les classes enfants, ou alors celle-ci doit aussi être abstraite.
- Vous pouvez également interdire l'héritage à l'aide du mot clé **final** sur une classe ou sur une méthode.